PROJECT DOCUMENTATION · FULL TECHNICAL REFERENCE

Lead Automation System

End-to-End AI-Powered Lead Pipeline

Automated enrichment · Claude AI report generation · WeasyPrint PDF rendering
SendGrid email delivery · Google Sheets & Drive integration

Tech Stack
Python 3.10+ · FastAPI · Anthropic Claude
WeasyPrint · SendGrid · Google APIs · Pydantic v2

Generated
May 21, 2026
github.com/Lalith9701/
lead-automation-system

Table of Contents

01 Project Overview

Lead Automation System is a fully automated, end-to-end pipeline that captures a prospect's company details via a web form, enriches the data from multiple sources, generates a personalised AI-powered PDF audit report using Anthropic Claude, and emails it directly to the prospect — all without any human intervention.

What Problem Does It Solve?

Sales and marketing teams spend hours manually researching prospects, writing personalised outreach, and creating custom reports. This system eliminates that entirely. The moment a prospect submits a form, the pipeline fires automatically and delivers a professional, data-rich audit report to their inbox within minutes.

Core Features

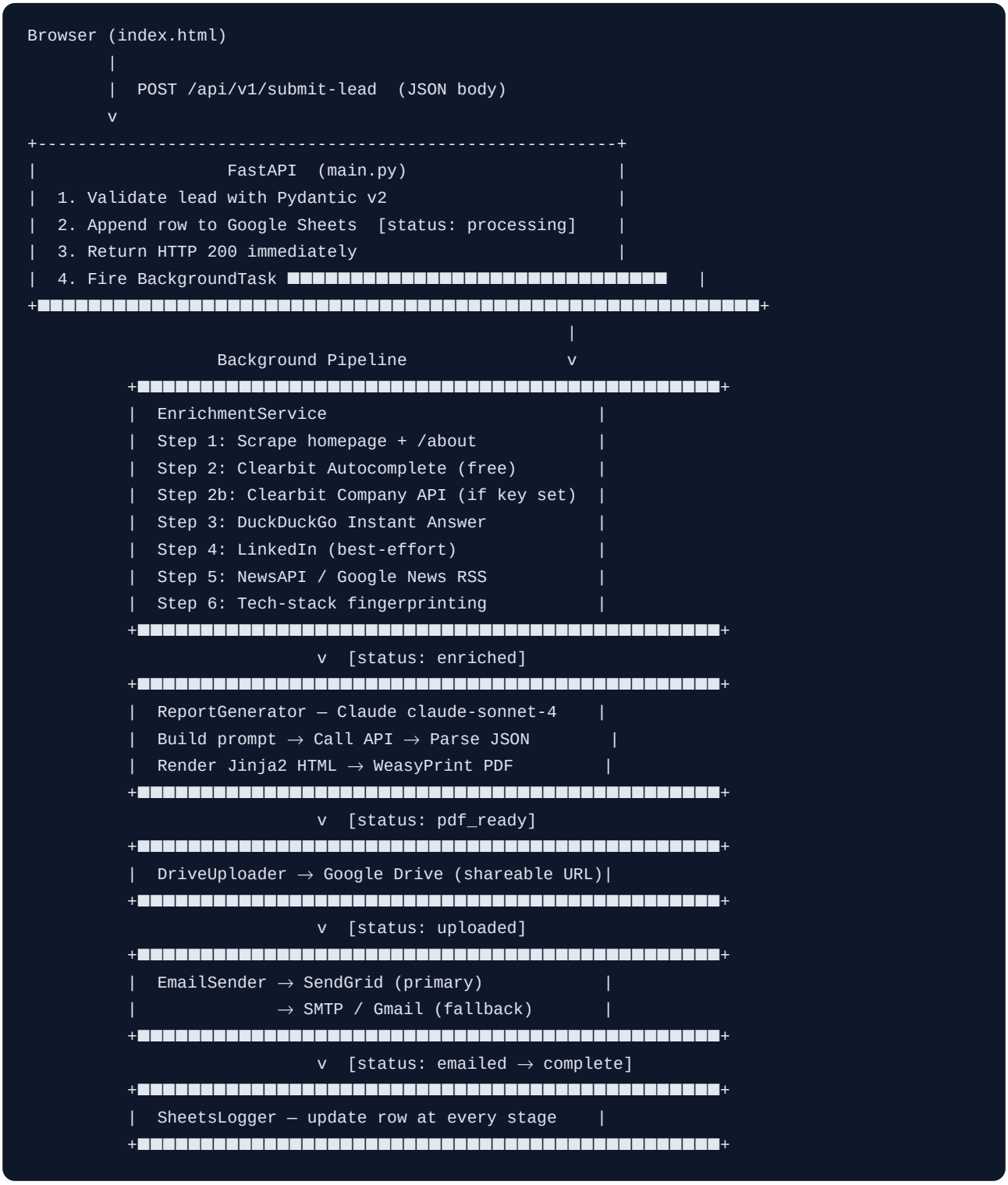
Feature	Description	Status
Lead Capture Form	Responsive HTML/CSS/JS form served at GET /	Live
Input Validation	Pydantic v2 with field-level error messages (422)	Live
Async Background Pipeline	FastAPI BackgroundTasks — instant 200 response	Live
Company Data Enrichment	6-source pipeline: scraping, Clearbit, DDG, LinkedIn, News, Tech	Live
AI Report Generation	Claude claude-sonnet-4 — 7-section structured JSON report	Live
PDF Rendering	Jinja2 HTML template → WeasyPrint PDF (10 pages)	Live
Email Delivery	SendGrid primary, SMTP fallback, HTML email + PDF attachment	Live
Google Sheets Logging	Real-time status tracking across all pipeline stages	Bonus
Google Drive Upload	PDF uploaded with public shareable link	Bonus
Swagger API Docs	Auto-generated at /docs and /redoc	Live

Key Design Principles

- **Never crashes** — every step is individually try/excepted; the pipeline always completes as many stages as possible
- **Instant response** — the API returns 200 immediately; all heavy work runs in the background

- **Graceful degradation** — if enrichment returns little data, Claude generates the best possible report from what's available
- **Zero external CSS frameworks** — the form and PDF use hand-written CSS for zero dependency overhead
- **Opt-in integrations** — Google Sheets, Drive, Clearbit, and NewsAPI all work without keys; they enhance but don't block

02 System Architecture



Pipeline Status Flow

processing	enriched	report_generated	pdf_ready	uploaded	emailed	complete

If any step fails, the status updates to <step>_failed with an error note in the Sheets row. The pipeline always continues to the next step.

03 Project Structure

```
lead-automation/
├── main.py                    # FastAPI app, static mount, health check
├── config.py                  # All env vars — single source of truth
├── requirements.txt           # 16 pinned dependencies
├── .env.example               # Template for .env — safe to commit
├── .gitignore                 # Excludes .env, credentials/, output/
├── README.md                  # Full project documentation
│
├── api/
│   ├── routes.py              # POST /api/v1/submit-lead + pipeline
│
├── models/
│   ├── lead.py                # LeadSubmission, LeadResponse (Pydantic v2)
│
├── services/
│   ├── enrichment.py          # EnrichmentService — 6-step pipeline
│   ├── report_generator.py    # Claude API + WeasyPrint PDF rendering
│   ├── email_sender.py        # SendGrid primary, SMTP fallback
│   ├── sheets_logger.py       # gspread real-time row updates [bonus]
│   ├── drive_uploader.py      # Google Drive upload + public link [bonus]
│
├── templates/
│   ├── report_template.html   # Jinja2 PDF template — 10 pages
│   ├── email_template.html    # Jinja2 HTML email template
│   ├── index.html             # Lead intake form — vanilla CSS/JS
│
├── static/
│   ├── styles.css             # WeasyPrint print overrides
│
└── utils/
    ├── helpers.py             # async_retry, safe_scrape, truncate_for_llm
    ├── scraper.py             # Homepage, DuckDuckGo, News, tech stack
```

File Responsibilities

File	Responsibility	~Lines
main.py	App factory, route registration, static files, health check	75
config.py	Centralised env var loading — all modules import from here	45
api/routes.py	Endpoint handler + full pipeline orchestration	130
models/lead.py	Input validation, URL normalisation, enum definitions	75

File	Responsibility	~Lines
services/enrichment.py	Multi-source enrichment, merge logic, confidence scoring	230
services/report_generator.py	Claude prompt building, JSON parsing, PDF rendering	210
services/email_sender.py	SendGrid + SMTP delivery, email template rendering	175
services/sheets_logger.py	Lazy gspread init, header management, real-time updates	120
services/drive_uploader.py	Drive API upload, public permission setting	75
utils/helpers.py	Retry decorator, safe HTTP, text truncation, domain extraction	90
utils/scraper.py	Homepage scraper, DuckDuckGo, Google News RSS, tech fingerprinting	140

04 Tech Stack

Layer	Library / Service	Version	Purpose
Web Framework	FastAPI	0.136+	Async REST API, BackgroundTasks, auto Swagger docs
ASGI Server	Uvicorn	0.47+	Production-grade ASGI server with hot reload
Validation	Pydantic v2	2.x	Request body validation, field-level errors, URL validation
AI / LLM	Anthropic Claude	claude-sonnet-4	7-section structured JSON report generation
PDF Generation	WeasyPrint	68+	HTML/CSS to PDF rendering (server-side, no headless Chrome)
Templating	Jinja2	3.x	PDF report template + HTML email template
HTTP Client	httpx	0.28+	Async HTTP for all external API calls and scraping
HTML Parsing	BeautifulSoup4 + lxml	4.14+	Homepage scraping, meta extraction, tech fingerprinting
Email (primary)	SendGrid SDK	6.12+	Transactional email with PDF attachment
Email (fallback)	smtplib (stdlib)	built-in	Gmail SMTP fallback when SendGrid unavailable
Google Sheets	gspread	6.2+	Lead logging and real-time status tracking
Google Drive	google-api-python-client	2.x	PDF upload with public shareable link
Config	python-dotenv	1.x	.env file loading

External APIs Used

API	Auth Required	Free Tier	Data Returned
Clearbit Autocomplete	No	Unlimited	Company name, domain, logo URL
Clearbit Company API	API Key	Limited	Full profile: founded, employees, HQ, industry
Clearbit Logo API	No	Unlimited	Company logo image
DuckDuckGo Instant Answer	No	Unlimited	Short company description / abstract
NewsAPI	API Key	100 req/day	Recent news articles with title, source, date
Google News RSS	No	Unlimited	Recent press mentions (RSS fallback)
Anthropic Claude	API Key	Pay per token	7-section structured JSON report
SendGrid	API Key	100 emails/day	Transactional email delivery
Google Sheets API	Service Acct	Yes	Lead logging and status tracking
Google Drive API	Service Acct	15 GB storage	PDF upload with shareable link

05 API Reference

Available Endpoints

Method	Path	Description
GET	/	Serve the lead intake HTML form
POST	/api/v1/submit-lead	Submit a lead — triggers the full automation pipeline
GET	/health	Health check — returns {status: ok, version: 1.0.0}
GET	/docs	Swagger UI — interactive API documentation
GET	/redoc	ReDoc — alternative API documentation
GET	/static/*	Static file serving (CSS, assets)

POST /api/v1/submit-lead — Request Fields

Field	Type	Required	Validation Rule
full_name	string	Yes	Min 2 characters, whitespace stripped
email	string	Yes	Valid email format (RFC 5322 via Pydantic EmailStr)
company_name	string	Yes	Min 1 character, whitespace stripped
company_website	string	Optional	Valid URL; https:// prepended automatically if missing
industry	string	Optional	Free text
company_size	enum	Optional	One of: 1-10, 11-50, 51-200, 201-500, 500+
role	string	Optional	Free text (e.g. CEO, Marketing Head)
message	string	Optional	Free text — pain points or goals

Responses

HTTP Code	When	Body
200 OK	Valid submission	{"status": "processing", "message": "Your report is being generated..."}

HTTP Code	When	Body
422 Unprocessable Entity	Validation failure	{"detail": [{"loc": ["body", "email"], "msg": "...", "type": "..."}]}
500 Internal Server Error	Unhandled exception	{"detail": "An internal error occurred. Please try again later."}

> The 200 response is returned immediately before the pipeline runs. Enrichment, AI generation, PDF rendering, and email delivery all happen asynchronously in the background. The prospect receives their email within 30–120 seconds depending on API response times.

06 Pipeline Deep Dive

The pipeline is orchestrated in **api/routes.py** inside the **_run_pipeline()** async function. It runs as a FastAPI BackgroundTask — completely decoupled from the HTTP request/response cycle.

Step 1 — Lead Capture & Validation

When a POST request arrives, FastAPI automatically deserialises the JSON body into a **LeadSubmission** Pydantic model. Validation runs synchronously before the handler executes. On failure, FastAPI returns a 422 with field-level error details. On success, the handler converts the model to a plain dict via **lead.model_dump()**, logs the initial row to Google Sheets, and returns 200 immediately.

Step 2 — Company Data Enrichment

The **EnrichmentService.enrich()** method runs 6 steps in sequence. Each step is wrapped in try/except — failure of any step is logged and skipped. Results are merged into a single enriched dict using **_merge()**, which only overwrites None or empty values (never clobbers good data with worse data).

#	Step	Source	Data Gathered	Key Required
1	Homepage Scrape	httpx + BS4	Title, meta description, H1/H2 headings, body text, /about text	No
2	Clearbit Autocomplete	Clearbit free API	Company name, domain, logo URL	No
2b	Clearbit Company API	Clearbit paid API	Founded year, employees, HQ, industry, social handles	Yes
3	DuckDuckGo Instant Answer	DDG API (free)	Short company description / abstract	No
4	LinkedIn Scrape	httpx (best-effort)	LinkedIn URL, tagline if not blocked	No
5	News Fetch	NewsAPI / RSS	Up to 5 recent news articles with title, source, date	Optional
6	Tech Stack Detection	HTML fingerprinting	Detected frameworks: React, Next.js, WordPress, etc. (21 patterns)	No

Step 3 — AI Report Generation

The **ReportGenerator.generate()** method builds a dynamic prompt combining lead data and enrichment data (truncated to 4,000 chars). Claude is instructed to respond with a specific 7-key JSON structure. The response is parsed with **_parse_json()** which strips markdown fences and tries substring extraction as a fallback. If parsing fails, the prompt is retried once with a stricter instruction. If it fails again, a minimal fallback report dict is used.

Steps 4–6 — PDF, Drive, Email

The PDF is rendered by loading the Jinja2 template, rendering it with the full context dict, then passing the HTML string to WeasyPrint. The PDF is saved to **output/reports/{company}_{timestamp}.pdf**. It is then optionally uploaded to Google Drive (if configured) and the shareable URL is included in the email. The email is sent via SendGrid with the PDF attached, falling back to SMTP if needed.

07 Data Models

LeadSubmission — Input Validation Model

```
class CompanySize(str, Enum):
    micro      = "1-10"
    small      = "11-50"
    medium     = "51-200"
    large      = "201-500"
    enterprise = "500+"

class LeadSubmission(BaseModel):
    full_name:       str                    # required, min_length=2
    email:           EmailStr               # required, RFC 5322 validated
    company_name:    str                    # required, min_length=1
    company_website: Optional[str] = None  # URL validated, https:// prepended
    industry:        Optional[str] = None
    company_size:    Optional[CompanySize] = None
    role:            Optional[str] = None
    message:         Optional[str] = None
```

Enrichment Data Dict — Internal Structure

```
enrichment_data = {
    "company_name":         str,
    "domain":               str | None,       # e.g. "acme.com"
    "description":          str | None,       # from meta tag or DuckDuckGo
    "industry":             str | None,
    "founded_year":         int | None,
    "employee_count":       str | None,       # e.g. "51-200"
    "hq_location":          str | None,       # e.g. "San Francisco, CA"
    "key_products_services": list[str],
    "recent_news":          list[dict],       # {title, source, published, url}
    "tech_stack":           list[str],        # e.g. ["React", "AWS", "HubSpot"]
    "social_media":         dict[str, str],   # {platform: url}
    "logo_url":             str | None,       # Clearbit Logo API URL
    "data_confidence":      str,              # "high" | "medium" | "low"
}
```

Report Content Dict — Claude Output Schema

```
report_content = {
    "executive_summary":      str,
    "company_overview": {
        "what_they_do":       str,
        "market_position":    str,
        "key_strengths":      list[str],
        "notable_achievements": str,
    },
    "digital_presence_audit": {
        "website_analysis":   str,
        "seo_observations":   str,
        "content_strategy":   str,
        "score":              str,   # e.g. "7/10"
        "recommendations":    list[str],
    },
    "competitive_landscape": {
        "market_overview":    str,
        "key_competitors":    list[str],
        "differentiators":    str,
        "threats_and_opportunities": str,
    },
    "growth_opportunities": [{
        "title":              str,
        "description":        str,
        "impact":             str,  # "High" | "Medium" | "Low"
        "effort":             str,  # "High" | "Medium" | "Low"
        "recommended_action": str,
    }],
    "tech_and_operations": {
        "current_stack_observations": str,
        "automation_gaps":    str,
        "recommended_tools":  list[str],
    },
    "action_plan": {
        "immediate_actions":       list[str],
        "short_term_30_60_days":   list[str],
        "long_term_90_plus_days":  list[str],
    },
    "closing_note":           str,
}
```

08 Enrichment Service

> The EnrichmentService runs 6 independent steps in sequence. Every step is wrapped in try/except — if one fails, the error is logged and the pipeline continues with whatever data was already gathered. Results from all steps are merged into a single dict.

_merge() Logic

The _**merge(base, update)** function is the backbone of the enrichment pipeline. It merges the update dict into base with these rules:

- **None / empty values** in update are skipped — never overwrite good data with nothing
- **Lists** are extended and deduplicated — tech stack items from multiple sources are combined
- **Dicts** are recursively merged — social media handles from different sources are combined
- **Scalars** only overwrite if the base value is falsy — first good value wins

Tech Stack Detection — 21 Fingerprints

Detection works by loading the homepage HTML and checking for known string patterns in script src attributes, meta tags, and inline content. No external service required.

WordPress	Shopify	Wix
Squarespace	Webflow	React
Next.js	Vue.js	Angular
jQuery	Bootstrap	Tailwind CSS
Google Analytics	Google Tag Manager	HubSpot
Intercom	Stripe	Cloudflare
AWS	Vercel	Netlify

Confidence Scoring

Level	Condition	Claude Behaviour
High	6 or more of 9 key fields populated	Full analysis with specific data references
Medium	3–5 of 9 key fields populated	Analysis with some assumptions noted
Low	0–2 of 9 key fields populated	Prompt instructs Claude to make clearly-labelled reasonable assumptions

09 AI Report Generation

System Prompt

```
You are a senior business analyst and consultant. You write professional,
insightful business audit reports that help companies understand their
digital presence, competitive position, and growth opportunities.
Your tone is authoritative but approachable. You must tailor every
section specifically to the company provided — never use generic filler text.
```

Prompt Construction

The user prompt is built dynamically in **_build_prompt()** and includes:

- Prospect name, role, company, website, industry, size
- Full enrichment data (truncated to 4,000 chars via truncate_for_llm())
- Prospect's free-text message (pain points / goals)
- Data confidence level with conditional instruction for low-confidence cases
- The exact JSON schema Claude must return — 7 top-level keys

JSON Parsing Strategy

Attempt	Method	Handles
1	Strip markdown fences, direct json.loads()	Clean JSON responses
2	Find outermost { } block, parse substring	JSON wrapped in prose
3	Retry Claude with stricter prompt instruction	Malformed first response
4	Return _fallback_report() — minimal safe dict	Complete Claude failure

Report Sections Generated by Claude

#	Section	Content
1	Executive Summary	2–3 paragraph personalised overview of the company
2	Company Overview	What they do, market position, key strengths, achievements
3	Digital Presence Audit	Website, SEO, content strategy, score /10, recommendations
4	Competitive Landscape	Market overview, competitors, differentiators, threats
5	Growth Opportunities	3–4 cards with Impact/Effort ratings and action steps

#	Section	Content
6	Tech & Operations	Stack observations, automation gaps, tool recommendations
7	Action Plan	Immediate / 30–60 day / 90+ day phased roadmap

10 PDF Generation

WeasyPrint converts HTML + CSS to PDF entirely server-side. No headless Chrome, no Puppeteer, no browser required. The Jinja2 template is rendered first, then the resulting HTML string is passed directly to WeasyPrint.

Rendering Pipeline

Step	What Happens
1. Context build	Dict assembled with lead, enrichment, report, generated_at, logo_url, static_dir
2. Jinja2 render	_jinja_env.get_template('report_template.html').render(**context) → HTML string
3. WeasyPrint	HTML(string=html_content, base_url=BASE_DIR).write_pdf(output_path)
4. CSS loading	static/styles.css loaded as WeasyPrint CSS object and passed as stylesheet
5. Output	Saved to output/reports/{company}_{YYYYMMDD_HHMMSS}.pdf

Report Template — 10 Pages

Page	Content	Key Elements
1	Cover Page	Full-bleed navy gradient, company logo, gold badge, prospect details
2	Table of Contents	7 numbered entries with descriptions, About This Report box
3	Executive Summary	Gold left-border callout box, key facts strip (industry, HQ, size)
4	Company Overview	2-column card grid, social links, product/service tags
5	Digital Presence Audit	Score circle, SVG bar chart, 3 analysis blocks, recommendations
6	Competitive Landscape	Market callout, competitor pill tags, differentiators vs threats
7	Growth Opportunities	Opportunity cards with Impact/Effort colour badges

Page	Content	Key Elements
8	Tech & Operations	2-column cards, detected tech stack tags, recommended tools
9	Action Plan	3-column colour-coded timeline, personal closing note, news feed
10	Disclaimer	Data sources table, legal notice, branded footer strip

WeasyPrint CSS Notes

- @page content rule adds running footer on every page (left: confidential label, right: page number + date)
- @page cover suppresses the footer on the cover page
- page-break-inside: avoid applied to all cards and multi-column blocks
- -webkit-print-color-adjust: exact forces background colours to render in PDF
- All CSS is inline in the template for maximum WeasyPrint compatibility
- SVG bar chart uses percentage-based widths — no external chart libraries needed

11 Email Delivery

The **EmailSender.send_report()** method tries SendGrid first. If SendGrid is not configured or returns an error, it automatically falls back to SMTP. Both paths use the same rendered HTML email body and PDF attachment.

Provider	When Used	Config Required	Free Tier
SendGrid	Primary — when SENDGRID_API_KEY is set	SENDGRID_API_KEY, FROM_EMAIL	100 e mails/ day
SMTP/Gma il	Fallback — when SendGrid fails or key missing	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD	Gmail free tier

Email Content

Element	Content
Subject	Your Personalized Business Audit Report — {company_name}
Greeting	Personalised with first name extracted from full_name
Teaser	First 300 chars of executive summary + first growth opportunity title
Digital score	Highlighted score from the digital presence audit section
CTA button	View Your Full Report — links to Google Drive URL if available
Attachment	PDF file attached directly to the email
Footer	Sender name, email, and disclaimer text

Gmail SMTP Setup

To use Gmail as the SMTP fallback, you must use an **App Password** (not your regular Gmail password). Enable 2-factor authentication on your Google account, then generate an App Password at **myaccount.google.com/apppasswords**. Use that 16-character password as SMTP_PASSWORD in your .env file.

SendGrid Sender Verification

SendGrid requires the FROM_EMAIL address to be verified as a sender. Go to **app.sendgrid.com** → **Settings** → **Sender Authentication** and verify your domain or single sender address before emails will be delivered.

12 Google Integrations

> Both Google Sheets logging and Google Drive upload are fully optional bonus features. The core pipeline works completely without them. They activate automatically when the relevant environment variables are set.

Google Sheets — Lead Logging Columns

Column	Field	Updated When
A	Timestamp	On lead submission
B	Full Name	On lead submission
C	Email	On lead submission
D	Company	On lead submission
E	Website	On lead submission
F	Industry	On lead submission
G	Role	On lead submission
H	Report Status	Updated at every pipeline stage
I	PDF Path	After PDF is rendered
J	Drive URL	After Drive upload
K	Enrichment Confidence	After enrichment completes
L	Error Notes	If any step fails

Google Drive — PDF Upload

The **DriveUploader** class uses the Google Drive v3 API to upload the generated PDF to a configured folder. After upload, it sets the file permission to **anyone with link can view** and returns the shareable URL. This URL is then included in the email as the CTA button link.

Service Account Setup — Step by Step

1. Go to Google Cloud Console → Create or select a project
2. Enable Google Sheets API and Google Drive API
3. Go to IAM & Admin → Service Accounts → Create service account
4. Download the JSON key file → save to ./credentials/service_account.json
5. Copy the service account email (e.g. name@project.iam.gserviceaccount.com)

6. Share your Google Sheet with that email (Editor role)
7. Share your Drive folder with that email (Editor role)
8. Set GOOGLE_SERVICE_ACCOUNT_JSON, GOOGLE_SHEET_ID, GOOGLE_DRIVE_FOLDER_ID in .env

13 Configuration & Environment Variables

All configuration is loaded in **config.py** via python-dotenv. Every other module imports from config — os.environ is never read directly anywhere else in the codebase.

Variable	Required	Default	Description
ANTHROPIC_API_KEY	Required	—	Anthropic API key. Get from console.anthropic.com
SENDGRID_API_KEY	Or SMTP	—	SendGrid API key. Get from app.sendgrid.com
FROM_EMAIL	Required	noreply@example.com	Verified sender email address
FROM_NAME	Required	Lead Automation	Sender display name in email
SMTP_HOST	Fallback	smtp.gmail.com	SMTP server hostname
SMTP_PORT	Fallback	587	SMTP port (587 for TLS)
SMTP_USER	Fallback	—	SMTP username / Gmail address
SMTP_PASSWORD	Fallback	—	SMTP password / Gmail App Password
CLEARBIT_API_KEY	Optional	—	Enables full Clearbit Company API enrichment
NEWS_API_KEY	Optional	—	NewsAPI key. Free tier: 100 req/day
GOOGLE_SERVICE_ACCOUNT_JSON	Bonus	./credentials/service_account.json	Path to Google service account JSON key
GOOGLE_SHEET_ID	Bonus	—	Google Sheet ID (from the URL)
GOOGLE_DRIVE_FOLDER_ID	Bonus	—	Google Drive folder ID for PDF uploads
PORT	Optional	8000	Server port for uvicorn

Variable	Required	Default	Description
OUTPUT_DIR	Optional	./output/reports	Directory where generated PDFs are saved

Minimum Required Configuration

To run the full pipeline end-to-end, you need at minimum:

```
ANTHROPIC_API_KEY=sk-ant-...        # for AI report generation
SENDGRID_API_KEY=SG....             # for email delivery
FROM_EMAIL=you@yourdomain.com       # verified sender address
FROM_NAME=Your Company Name
```

Everything else is optional and degrades gracefully when not set. The enrichment pipeline, form, and PDF rendering all work without any optional keys.

14 Error Handling & Resilience

> The system is designed to never fully crash. Every external call is individually wrapped in try/except. If a step fails, the error is logged, the Sheets row is updated with the failure reason, and the pipeline continues to the next step.

async_retry Decorator

Located in **utils/helpers.py**, the @async_retry decorator wraps any async function with exponential back-off retry logic:

```
@async_retry(retries=3, delay=1.0)
async def my_api_call():
    ...

# Retry schedule:
# Attempt 1 → wait 1s → Attempt 2 → wait 2s → Attempt 3
# If all 3 fail, the last exception is re-raised
```

safe_scrape Helper

All web scraping goes through **safe_scrape(url)** which:

- Sets a 10-second timeout on every request
- Sends browser-like headers (User-Agent, Accept, Accept-Language) to reduce blocking
- Follows redirects automatically
- Returns None on any exception — never raises, never crashes the pipeline

Failure Behaviour Per Pipeline Step

Step	On Failure	Pipeline Continues?
Enrichment (any sub-step)	Log warning, skip that source, continue with partial data	Yes
Enrichment (entire service)	Log error, use empty enrichment dict, update Sheets	Yes
Claude API call	Retry once with stricter prompt, then use fallback report dict	Yes
JSON parsing	Try substring extraction, retry Claude, use fallback dict	Yes
PDF rendering	Log error, skip email step, update Sheets as pdf_failed	Partial

Step	On Failure	Pipeline Continues?
Drive upload	Log warning, continue without Drive URL in email	Yes
SendGrid email	Fall back to SMTP automatically	Yes
SMTP email	Log error, mark email_failed in Sheets, do not re-raise	Yes
Sheets logging	Log warning, silently skip — never blocks pipeline	Yes

truncate_for_llm

Scraped text is truncated before being sent to Claude using **truncate_for_llm(text, max_chars=3000)**. It breaks at a word boundary (not mid-word) and appends '… [truncated]'. This prevents token overflow and keeps API costs predictable.

15 Running the Project

1. Clone the Repository

```
git clone https://github.com/Lalith9701/lead-automation-system.git
cd lead-automation-system
```

2. Create Virtual Environment

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. WeasyPrint System Dependencies

```
# Windows — install GTK3 runtime first:
# https://github.com/tschoonj/GTK-for-Windows-Runtime-Environment-Installer

# macOS
brew install pango

# Ubuntu / Debian
sudo apt-get install libpango-1.0-0 libpangoft2-1.0-0
```

5. Configure Environment

```
# Windows
copy .env.example .env

# macOS / Linux
cp .env.example .env

# Then edit .env and fill in your API keys
```

6. Run the Server

```
uvicorn main:app --reload --port 8000
```

7. Access the Application

URL	Description
http://localhost:8000	Lead intake form
http://localhost:8000/docs	Swagger UI — interactive API docs
http://localhost:8000/redoc	ReDoc API documentation
http://localhost:8000/health	Health check — returns {status: ok, version: 1.0.0}

8. Test the Pipeline via curl

```
curl -X POST http://localhost:8000/api/v1/submit-lead \
  -H "Content-Type: application/json" \
  -d '{
    "full_name": "Jane Smith",
    "email": "jane@acme.com",
    "company_name": "Acme Corp",
    "company_website": "https://acme.com",
    "industry": "SaaS",
    "company_size": "51-200",
    "role": "CEO",
    "message": "We want to improve our SEO."
  }'

# Expected response:
{"status": "processing", "message": "Your report is being generated..."}
```

After submitting, watch the server logs — you will see each pipeline step execute in real time. The generated PDF will appear in output/reports/.

16 Assumptions & Tradeoffs

Design Assumptions

Area	Assumption	Rationale
Clearbit free tier	Autocomplete endpoint always called without a key	Publicly accessible, returns useful logo + domain data
LinkedIn scraping	Best-effort only — expected to fail most of the time	LinkedIn aggressively blocks bots; pipeline continues gracefully
WeasyPrint CSS	All CSS is inline in the template	WeasyPrint has limited external stylesheet support
Claude model	claude-sonnet-4-20250514 used by default	Best balance of quality and speed; configurable in config.py
Output directory	./output/reports/ created automatically on startup	Avoids manual setup step for new deployments
Google features	Fully opt-in — pipeline works without Google credentials	Reduces setup friction for users who don't need logging/Drive
Background tasks	FastAPI BackgroundTasks used for pipeline execution	Simple, in-process, zero infrastructure — good for low-medium volume
PDF storage	PDFs saved to local filesystem	Simple default; replace with S3 for production scale

Tradeoffs

Decision	Benefit	Limitation
WeasyPrint over headless Chrome	No browser dependency, lighter, faster startup	Limited CSS support — flexbox/grid partially supported
Single Claude call + 1 retry	Fast, low cost, simple	Complex JSON can occasionally fail parsing on first attempt
Multi-source enrichment	More data, higher confidence scores	Slower (3–8s total); all steps are async to mitigate
SendGrid → SMTP fallback	Reliable delivery with automatic failover	SMTP requires Gmail App Password setup

Decision	Benefit	Limitation
FastAPI BackgroundTasks	Zero infrastructure, simple deployment	Tasks lost on server restart; use Celery+Redis for production
Local PDF storage	No cloud setup required	Not suitable for multi-instance deployments

How to Scale to Production

Component	Current	Production Recommendation
Task queue	FastAPI BackgroundTasks	Celery + Redis or AWS SQS
PDF storage	Local filesystem	AWS S3 or Google Cloud Storage
State persistence	Google Sheets (optional)	PostgreSQL with SQLAlchemy
Rate limiting	None	slowapi middleware on /submit-lead
Authentication	None (open endpoint)	API key header or OAuth2
Monitoring	Python logging to stdout	Sentry for errors, Datadog for metrics
Deployment	uvicorn locally	Docker + Gunicorn + Nginx or AWS ECS
Secrets	.env file	AWS Secrets Manager or HashiCorp Vault

Extending the System

- **New enrichment source:** Add a method to EnrichmentService, call it in enrich(), merge with _merge()
- **New email provider:** Add _send_<provider>() to EmailSender, update dispatch logic in send_report()
- **New report section:** Extend JSON schema in _build_prompt(), add HTML block to report_template.html
- **CRM webhook:** Add a step in _run_pipeline() after email to POST lead data to HubSpot/Salesforce
- **Multi-language reports:** Pass a language field in the lead form and include it in the Claude system prompt
- **Report caching:** Hash the company domain and cache enrichment data in Redis to avoid re-scraping

Quick Reference Summary

Core Pipeline

1. Lead form submission
2. Pydantic v2 validation
3. Instant 200 response
4. Company enrichment (6 sources)
5. Claude AI report generation
6. WeasyPrint PDF rendering
7. Google Drive upload
8. SendGrid / SMTP email
9. Sheets status logging

Key Files

main.py — App entry point
config.py — All settings
api/routes.py — Endpoint + pipeline
models/lead.py — Validation
services/enrichment.py
services/report_generator.py
services/email_sender.py
templates/report_template.html
utils/helpers.py — Retry / scrape

Minimum Config

ANTHROPIC_API_KEY
→ `AI report generation`

SENDGRID_API_KEY
→ `Email delivery`

FROM_EMAIL
→ `Verified sender`

FROM_NAME
→ `Sender display name`

Start the Server

```
uvicorn main:app --reload --port 8000
```

Lead Automation System
Automated · AI-Powered · Personalised
github.com/Lalith9701/lead-automation-system

Generated
May 21, 2026
Full Technical Reference
16 Sections · ReportLab PDF